

# OPWA Software Requirement Specifications

## HCMC Metro Operation Web Application

### For Admins, Operators and Ticket Agents

## Table of Contents

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Introduction .....</b>                             | <b>3</b> |
| <b>I. Black Pepper Level .....</b>                    | <b>3</b> |
| I.G. General Requirements .....                       | 3        |
| I.G.1. Frontend Architecture .....                    | 4        |
| I.G.2. Backend Architecture .....                     | 4        |
| I.G.3. Frontend Utility Class for REST Requests ..... | 4        |
| I.G.4. Global URL Configuration .....                 | 5        |
| I.OP. OPWA System Requirements .....                  | 5        |
| I.OP.1. Staff Account Creation .....                  | 5        |
| I.OP.2. Authentication & Authorization .....          | 6        |
| I.OP.3. Manage Metro Lines .....                      | 6        |
| I.OP.4. Staff Profile .....                           | 7        |
| I.OP.5. Purchasing Tickets for Passengers .....       | 7        |
| I.OP.6. View Statistics of Ticket Purchase .....      | 8        |
| <b>II. Habanero Level .....</b>                       | <b>8</b> |
| II.G. General Requirements .....                      | 8        |
| II.G.1. Frontend Architecture .....                   | 8        |
| II.G.2. Backend Architecture .....                    | 9        |
| II.G.2. HTTPS Enforcement .....                       | 10       |
| II.G.3. Cross-Origin Resource .....                   | 10       |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| II.G.4. Responsiveness .....                                         | 11        |
| II.OP. OPWA System Requirements .....                                | 11        |
| II.OP.1. Validation of Staff Data .....                              | 11        |
| II.OP.2. Authentication & Authorization .....                        | 13        |
| II.OP.3. Create Metro Line with Scheduling Information .....         | 14        |
| II.OP.4. Cash and Credit Card Payment .....                          | 15        |
| II.OP.5. Purchasing Tickets with Persistent Ticket Cart .....        | 16        |
| II.OP.6. Real-Time Metro Line Suspension Notification .....          | 17        |
| <b>III. Red Savina Level .....</b>                                   | <b>18</b> |
| III.G. Modularized Architecture for Reusability .....                | 18        |
| III.G.1. Frontend Architecture .....                                 | 18        |
| III.G.2. Backend Architecture.....                                   | 19        |
| III.OP. OPWA Advance System Requirements.....                        | 20        |
| III.OP.1. Allowing Adding Images of National ID and Student ID ..... | 20        |
| III.OP.2. Allow Staff to Login with Google Account .....             | 20        |
| III.OP.3. Advance Metro Lines Search API .....                       | 21        |
| III.OP.4. Route Visualization on Map API .....                       | 22        |
| III.OP.4. Anonymize Sensitive Staff Data .....                       | 22        |
| III.OP.5. Re-upload National ID and/or Student ID Proof.....         | 22        |
| III.OP.6. Dynamic, Informative, and Persisted Ticket Cart.....       | 23        |
| III.OP.7. Partition and Shard Staff Data .....                       | 23        |
| <b>IV.OPAC. OPWA API Consumption Requirements .....</b>              | <b>24</b> |
| IV.OPAC.1. Payment API .....                                         | 24        |
| IV.OPAC.2. Ticket Types API .....                                    | 25        |
| <b>V.OPAP. OPWA API Provision Requirements .....</b>                 | <b>25</b> |
| V.OPAP.1. Metro Line Data Search and Listing API .....               | 25        |
| V.OPAP.2. Real-Time Metro Line Suspension Notification API .....     | 25        |

## Introduction

The purpose of this project is to develop two integrated web applications to support the successful implementation of the Ho Chi Minh City (HCMC) Metro system. As HCMC embarks on an ambitious plan to construct 7 metro lines by 2035, these applications will address both passenger and operator needs. The first web application will enable passengers to search for metro routes, book tickets, and access metro services directly from their mobile phones or PCs. The second web application will support metro operators in managing stations, tracking delays, and analyzing traffic statistics to optimize schedules and service efficiency. Ultimately, this software will facilitate smoother urban transport, reduce traffic congestion, and contribute to the city's economic and environmental well-being.

### System Overview

This project involves the development of two subsystems, each serving a distinct user group. **Team OPWA** (OPerator Web Application) will develop the web application for metro operators, enabling them to manage metro lines, monitor station status, update delays, and view real-time traffic data to adjust schedules as necessary. **Team PAWA** (PAssenger Web Application) is responsible for the development of the web application for passengers. This application will allow users to search for routes, book tickets, and use digital ticket scanning for entry into the metro system.

Both subsystems must be designed to communicate seamlessly with each other. The specific requirements for data exchange, including the communication channels and data structures, will be outlined in the corresponding sections of the document.

The software requirements will be categorized into three levels, inspired by the Scoville scale of spiciness — **Black Pepper**, **Habanero**, and **Red Savina** —to reflect the complexity and progression of development. These levels will help guide both teams in managing and prioritizing tasks throughout the project.

## I. Black Pepper Level

### I.G. General Requirements

These requirements apply to both Team **PAWA** and Team **OPWA**.

### I.G.1. Frontend Architecture

- The frontend application must implement an **N-Tier architecture**.
  - For **React applications**, the architecture must include at least the following three tiers:
    - **Page Tier**: Responsible for rendering pages and managing routing.
    - **UI Components Tier**: Contains reusable user interface components.
    - **Backend API Calls Tier**: Handles all interactions with backend APIs.
  - For **Vanilla JavaScript applications**, the architecture must include at least the following three tiers:
    - **Presentation Tier**: Manages the user interface and rendering of content.
    - **Event Handler Tier**: Handles user interactions and events.
    - **Backend API Call Tier**: Manages all communication with the backend services.

### I.G.2. Backend Architecture

- The backend application must be developed using **Spring Boot** and follow an **N-Tier architecture** with at least four tiers:
  - **Controller Tier**: Handles HTTP requests and routes them to the appropriate services.
  - **Service Tier**: Contains business logic and processes requests from the controller.
  - **Repository Tier**: Manages data persistence and database operations.
  - **Model Tier**: Defines the data structures used throughout the application.

### I.G.3. Frontend Utility Class for REST Requests

- To enhance **usability** and **maintainability**, the frontend application must include a **Utility class** that:
  - Centralizes the logic for sending REST requests to the backend.
  - Handles request headers, authentication tokens, and error handling consistently.

- Ensure that modifications to REST interactions can be made in a single location, reducing duplication.

#### I.G.4. Global URL Configuration

- The frontend application must maintain a **Global URL configuration** file that:
    - Contains all backend endpoint definitions.
    - Allows easy modification of endpoint patterns from a single source, minimizing the effort required to update endpoints across the application.
- 

### I.OP. OPWA System Requirements

#### I.OP.1. Staff Account Creation

The Master Admin shall provide a link to the staff for creating their accounts with the following attributes:

- Username
- Email
- Password
- First Name
- Middle Name
- Last Name
- National ID
- Role (Admin, Operator, or Ticket Agent)
- Residence Address
- Phone Number
- Date of Birth
- Employed:
  - **True:** the staff is working at HCMC Metro
  - **False:** the staff is no longer working at HCMC Metro
- Shift: staff can work in one of the three shifts:

- **Day:** 06:00 AM – 02:00 PM
- **Evening:** 02:00 PM – 10:00 PM
- **Night:** 10:00 PM – 06:00 AM

A **Master Admin account** shall be created manually in the database with the same attributes as other staff members.

---

## I.OP.2. Authentication & Authorization

**Attention: If you aim for the Red Savina level, you shall read III.OP.2, as OPWA are required to login using OAuth 2 on Google Accounts, not using local authentication solution.**

The OPWA system shall implement user authentication using Basic Authentication. The system must:

- Authenticate users by verifying the provided email address and password against stored credentials.
  - Utilize Bcrypt to securely compare hashed passwords during authentication.
  - Deny access if authentication fails and provide appropriate error messages.
  - Enforce the following role-based access control:
    - **Admins:** Access all pages in the OPWA system.
    - **Operators:** Access all pages in the OPWA system.
    - **Ticket Agents:** Access only the staff profile, ticket purchase, and passenger purchase history pages.
- 

## I.OP.3. Manage Metro Lines

Operators shall have full control over metro line management, including:

- Creating, reading, updating, and deleting metro lines and stations.
- Specifying the following attributes when creating a metro line:
  - Line Name
  - Station Names (ordered list, determining the metro line path)
  - Total travel duration (in minutes)

By default, the Metro Line Management page shall display only key details:

- Metro Line Name
- Start Station
- End Station
- Total Duration

Upon user interaction (clicking on a metro line), the system shall expand to display all intermediate stations in the correct order.

---

### I.OP.4. Staff Profile

Each staff member shall have access to a profile page displaying the details provided during registration, as defined in **I.OP.1**.

---

### I.OP.5. Purchasing Tickets for Passengers

Ticket Agents shall be able to purchase tickets for guests or registered passengers using their:

- **National ID** (for guests)
- **Passenger ID** (for registered users, retrieved from PAWA system)

#### **Ticket Type Selection**

- The six available ticket types shall be retrieved dynamically from the PAWA system. (Refer to **I.PA.6** in PAWA SRS.)

#### **Payment Methods**

- Ticket Agents can purchase tickets using:
  - **Passenger's e-Wallet** (for registered passengers)
  - **Cash Payment** (for both guests and registered passengers)

#### **Cash Payment Handling**

- If a passenger pays by cash, the Ticket Agent must enter the received amount.
- OPWA shall calculate and display the **change amount** for convenience.

**Example:**

- A Daily Ticket costs **40,000 VND**.
- The passenger pays **50,000 VND**.
- The system must display a change amount of **10,000 VND**.

This feature ensures Ticket Agents can efficiently handle transactions and provide accurate change to passengers.

---

### I.OP.6. View Statistics of Ticket Purchase

Operators shall have access to ticket sales statistics, including:

- **Total purchase value** (in VND)
- **Total number of tickets sold**
- **Statistics covering all metro lines** from the launch of the system

This feature provides essential insights for operators to analyze ticket sales trends and passenger demand.

---

## II. Habanero Level

### II.G. General Requirements

The following requirements apply to both **PAWA** and **OPWA** teams to ensure architectural consistency, security, and interoperability.

#### II.G.1. Frontend Architecture

- **Separation of Concerns:** UI components must adhere to a modular design, separating **API communication** from **event handling** and **presentation logic** to enhance maintainability and scalability.
- **Component Abstraction in React:**
  - UI elements must be structured into **Pages** and **Components**:

- **Pages:** High-level containers that compose multiple UI components to form a complete interface (e.g., a **Dashboard Page** includes a **Search Bar** for metro stations and a **Carousel** for announcements).
  - **Components:** Reusable UI elements that encapsulate a specific feature (e.g., **Carousel**, **Table**, **Search Bar**, **Button**).
  - **State Management:**
    - Utilize **local component state** for isolated interactions and **global state management** (e.g., Redux, React Context) for shared data across components.
- 

## II.G.2. Backend Architecture

The **Spring Boot** application must follow a **modular architecture** with the following constraints to enhance maintainability, scalability, and separation of concerns:

- **Layered Structure for Each Module:**
  - Each module must have **at least four layers**:
    1. **Controller Layer:** Handles incoming HTTP requests and responses.
    2. **Service Layer:** Implements business logic and interacts with repositories.
    3. **Repository Layer:** Manages database operations using JPA/Hibernate.
    4. **Model Layer:** Defines the domain objects/entities for the module.
  - **Modules must be dedicated to specific business capabilities.**
    - Example: The **PaymentModule** should handle only payment-related services and must not contain business logic related to ticketing or user management.
- **Encapsulation through Service Interfaces:**
  - Each module must provide a **Service Interface** that **exposes only necessary functions** for external modules.
  - Other modules must **only interact with the exposed Service Interface** and must not directly access internal Service layer functions.

- Example:
  - The **TicketModule** may expose a `getTicketDetails(ticketId)` function via its service interface.
  - The `PaymentModule` should **only access this function** instead of querying the Ticket database directly.
- **Data Transfer Object (DTO) Usage:**
  - Modules must define **DTOs to control data exposure**.
  - A DTO should represent **only a subset** of the Model, ensuring that unnecessary attributes **are not exposed** to the frontend or external modules.
  - Example:
    - A **UserDTO** may expose only `userId`, `fullName`, and `email`, while keeping `passwordHash` and `userRoles` hidden.

By enforcing this structured **modular backend architecture**, PAWA and OPWA will achieve:

- **Better maintainability** through clear separation of responsibilities.
- **Stronger security** by restricting direct access between modules.
- **Improved scalability** as new modules can be integrated with minimal impact on existing ones.

---

## II.G.2. HTTPS Enforcement

- All backend and frontend communications must be secured using **HTTPS (HyperText Transfer Protocol Secure)** to prevent data interception and unauthorized access.
- Each team must generate **self-signed asymmetric key pair(s)** to enable HTTPS protocol on **the backend and frontend** services.

---

## II.G.3. Cross-Origin Resource Sharing (CORS) Policy

- Each backend service must enforce a **restrictive CORS policy** to allow requests **exclusively from the PAWA and OPWA frontend applications** rather than using wildcard (\*) for trusted origins.

## II.G.4. Responsiveness

The OPWA web application UI must be **responsive to Operators**, ensuring usability on **both mobile and PC devices**. **Ticket Agent and Admin UIs are optimized for desktop usage; thus, their UI must be responsive only on desktop and laptop devices**. The system shall:

- Utilize **responsive design principles**, ensuring all UI components adjust dynamically based on screen size and resolution.
- Implement a **mobile-friendly layout for Operator UIs**, where navigation, buttons, and forms remain accessible and user-friendly on smaller screens.
- Support **fluid resizing**, ensuring that content and interactive elements maintain usability when the browser window is resized.

This requirement ensures accessibility and usability for the users across different devices, improving operational efficiency for metro staff.

---

## II.OP. OPWA System Requirements

### II.OP.1. Validation of Staff Data

The OPWA shall enforce strict validation rules for staff data to ensure data integrity and security. Each input field must adhere to the following validation criteria:

- **Email Address:**
  - Must follow a valid email format (e.g., example@domain.com).
  - Must end with **.com** or **.vn** domain extensions.
  - Must not contain spaces or special characters other than @ and .
  - Must be unique within the system.
- **Password:**
  - Must be hashed using Bcrypt before storage.
  - Must be at least **8 characters** long.
  - Must contain at least **one uppercase letter, one lowercase letter, and one digit, and one special character** (e.g., @, #, \$, %)

- **First Name, Middle Name, and Last Name:**
  - Must support both English and Vietnamese alphabet characters, including diacritics (e.g., à, Ô, ê, Û, ơ).
  - Must not contain numbers or special characters.
  - Maximum length: **50 characters** per field.
- **National ID:**
  - Must be exactly **12 digits** (Vietnamese Citizen ID format).
  - Must consist of only numeric characters.
  - Must be unique within the system.
- **Date of Birth:**
  - Must follow the format **dd/mm/yyyy** (e.g., 15/08/2000).
  - Must represent a valid calendar date (e.g., 30/02/2000 is invalid).
  - Must indicate an age of at least **6 years old**.
- **Residence Address:**
  - Must be split into Number, Street, Ward, District, and City
  - Must not contain special symbols except , . - /
- **Phone Number:**
  - Must follow the Vietnamese phone number format.
  - Must contain exactly **10 digits**, starting with 0.
- **Employed:** True or False
- **Role:**
  - Must be one of the following options: **Admin**, **Operator**, or **Ticket Agent**.
- **Shift:**
  - Staff must only work in one of the following shifts:
    - **Day:** 06:00 AM – 02:00 PM
    - **Evening:** 02:00 PM – 10:00 PM
    - **Night:** 10:00 PM – 06:00 AM

**Each field has a placeholder value with an example of a valid input.** For example, **Phone number's placeholder** is 0921 123 456; **Date of birth's placeholder** is 21/12/1995.

The system shall **reject any registration attempt** that fails to meet the above validation criteria and return appropriate error messages specifying the reason for rejection.

The error message must clearly **explain the cause of the validation failure** and **provide an example of a valid input**. For instance, if an invalid email is provided, the system shall return:

**"Invalid email format. The email must end with '.com' or '.vn' (e.g., example@domain.com)."**

Similarly, if an incorrect date format is entered, the system shall return:

**"Invalid date of birth format. Please use 'dd/mm/yyyy' (e.g., 15/08/2000)."**

This ensures users receive clear feedback, enabling them to correct their input efficiently.

## II.OP.2. Authentication & Authorization

**Attention: If you aim for the Red Savina level, you shall read III.OP.2 first, as OPWA are required to login using OAuth 2 on Google Accounts, not using local authentication solution as II.OP.2 requires.**

The OPWA backend shall **implement authentication using JSON Web Tokens (JWT)**. Upon receiving valid credentials (a registered email and the correct password), the system must generate a signed JWT token containing user identification details and authorization claims.

- The JWT must be used in authorizing OPWA and PAWA users **in both the frontend and backend of OPWA**:
  - **At minimum**, the JWT token should be included in the **Authorization** header of every request that requires authentication.
  - The OPWA frontend shall store the token securely and include it in all protected API requests. **The token must NOT be stored in local storage or session storage to prevent vulnerabilities such as cross-site scripting (XSS).**
  - The OPWA backend shall validate the JWT in every request to ensure that the OPWA user is authorized to access the requested resource. **Tokens with**

**invalid signatures or expired timestamps must be rejected, and an appropriate error response shall be returned to the OPWA user.**

- When receiving requests from PAWA, the JWT generated by PAWA in the request must be verified:
    - The OPWA backend shall verify PAWA JWT's **signature**, and **expiration time** before granting access to requested resources.
    - If the PAWA token is invalid or unauthorized, OPWA must return an HTTP 401 Unauthorized response with an error message specifying the reason for rejection.
- 

## II.OP.3. Create Metro Line with Scheduling Information

### Metro Line Creation and Scheduling

In addition to the requirements of **I.OP.3**, OPWA must allow Operators to define metro lines with detailed scheduling information. When creating a metro line, Operators must specify:

- **Line Name**
- **List of Stations in Cardinal Order**
- **Total Travel Duration (in minutes)**
- **First Departure Time of the Day**
- **Train Frequency (Interval between departures, in minutes)**

### Automated Trip Generation

Upon metro line creation, OPWA shall automatically generate trips based on the provided schedule. The system shall:

- Compute **trip schedules** for the entire metro line, ensuring that each train follows the predefined station order and travel duration.
- Generate a **full-day schedule** of trips based on the first departure time and frequency.
- Ensure that all trips correctly reflect **stop times at each station**, allowing PAWA to retrieve and display accurate scheduling data to passengers.

For example, if an Operator defines a metro line with a **first departure at 05:30 AM**, a **10-minute interval**, and a **30-minute travel duration**, OPWA shall generate trips with traveling times such as **05:30 - 06:00 AM**, **05:40 - 06:10 AM**, **05:50 - 06:20 AM**, and so on throughout the operational hours from **05:30 AM to 10:00 PM**. The last trip starts at 10:00 PM.

### Schedule Overview and Lazy Loading

- After automatically generating the trips, OPWA must provide a **schedule overview** displaying the **first two** and **last two** trips in the schedule.
- Example Schedule Overview:
  - **First two trips: 05:30 - 06:00 AM, 05:40 - 06:10 AM**
  - **Last two trips: 09:50 - 10:20 PM, 10:00 - 10:30 PM**
- A **complete list** of all generated trips shall be provided after the schedule overview. This list shall be **lazily loaded** as Operators scroll down, ensuring a smooth and responsive experience.

### Metro Line Management Interface

- The Metro Line Management page must provide an **intuitive UI** for Operators to enter schedule parameters when creating or modifying a metro line.
- The system shall allow Operators to **view, update, or delete** previously created schedules.
- If a schedule is **modified**, the system must update all **future trips** accordingly, while **past trip records remain unchanged**.

### Data Integration with PAWA

- OPWA shall provide PAWA access to **up-to-date metro line schedules** and routes through an API.
- The PAWA team must **not** implement backend features for **creating or managing metro lines**.

---

## II.OP.4. Support Cash Payment at Ticket Booths

In addition to the requirements of **I.OP.4**, OPWA must support **Ticket Agent** process ticket purchases by cash using **the PAWA Payment API**.

- **Cash Payment:**
  - Both **guests** and **authenticated passengers** can purchase metro tickets using cash.
  - Authenticated passengers can disclose their passenger ID to pay the ticket by their wallet. **Bonus points will be given if passengers can show a QR code and the OPWA system automatically checks if the wallet money of the passenger is sufficient to pay for the ticket.**
  - Payments must be processed via the **PAWA Payment API**.
  - Upon successful payment, PAWA must confirm the success of the transaction.
  - If the payment fails, PAWA must report the error back to OPWA.

## II.OP.5. Purchasing Tickets with Persistent Ticket Cart

In addition to the requirements specified in I.OP.5, **OPWA** must implement a **Ticket Cart system** to enhance the efficiency of ticket purchases made by **Ticket Agents** on behalf of passengers. The Ticket Cart must provide the following features:

- **Multi-Ticket Selection:** Ticket Agents can add multiple ticket types and quantities to the cart before completing the purchase. For example, an agent can add **one Monthly Ticket for Students and two Three-Day Tickets** in a single transaction for a passenger.
- **Cart Management:** The cart must support functionalities to **remove individual tickets or clear all items** before finalizing the purchase.
- **Real-Time Pricing Information:** The cart must display the **subtotal cost for each ticket type and the total cost of all selected tickets**. For example, if a **Monthly Ticket for Students** costs **150,000 VND** and **two Three-Day Tickets** cost **180,000 VND**, the cart must show both **subtotals and a final total of 330,000 VND** before checkout.
- **Persistent Access Across OPWA Pages:** The Ticket Cart must be accessible from any page within **OPWA**, allowing **Ticket Agents** to view or modify the cart without interrupting their workflow.
- **Session Persistence:** The cart must retain its contents when **Ticket Agents** navigate between different **OPWA** web pages, ensuring that selected tickets are not lost until explicitly removed or purchased.

- **Integration with PAWA Payment Service: OPWA must utilize the PAWA payment service to conduct ticket transactions.** The OPWA team must not develop an independent payment service or implement transaction recording within OPWA. Instead, **all payments and transaction logs must be handled through PAWA's existing payment system** to maintain system consistency and centralized financial recordkeeping.

This feature aims to provide **Ticket Agents** with a **seamless, efficient, and flexible ticket purchasing process**, ensuring ease of access, real-time pricing visibility, and an improved passenger experience while maintaining integration with PAWA's financial infrastructure.

## II.OP.6. Real-Time Metro Line Suspension Notification

In addition to the requirements specified in **I.OP.3 (Manage Metro Lines)**, OPWA must provide real-time notifications to the PAWA system when an Operator marks a metro line or segment as suspended due to emergencies or maintenance. This ensures that passengers using PAWA receive timely updates about disruptions.

### Trigger Conditions:

- When an Operator marks a **metro line** or a **specific segment** as suspended due to:
  - **Emergency situations** (e.g., accidents, security incidents).
  - **Scheduled maintenance** (e.g., track repairs, system upgrades).

### System Behavior:

- **Immediate Notification:** Upon suspension, OPWA must generate and send an event-driven notification to PAWA in real time.
- **Notification Data:** The notification must contain:
  - **Metro Line and Station Details:** Affected metro line and specific stations impacted.
  - **Reason for Suspension:** Emergency or maintenance, with a brief description.
  - **Expected Restoration Time:** If available, OPWA should include the estimated time when the service will resume.

### Delivery & Integration:

- **Low-Latency API Transmission:** The notification must be transmitted via OPWA's real-time API to PAWA for immediate processing.

- **In-App Passenger Alerts:** PAWA must use this information to display alerts to passengers currently using the app, minimizing inconvenience.

#### System Acknowledgment:

- **Acknowledgment to PAWA:** OPWA must wait for confirmation from PAWA that it successfully processes the suspension notification. PAWA must explicitly acknowledge successful processing of the suspension notification to OPWA. This acknowledgment ensures the notification is not resent unnecessarily.
- **Retry Notification:** If OPWA does not receive a confirmation from PAWA that the suspension notification has been successfully processed, OPWA must automatically resend another notification **three seconds** after detecting a timeout or failure.

## III. Red Savina Level

### III.G. Modularized Architecture for Reusability

The following requirements apply to both PAWA and OPWA teams to ensure a modularized architecture in both the frontend and backend, fostering reusability, maintainability, and scalability.

#### III.G.1. Frontend Architecture

The frontend application must implement a **componentized frontend architecture** with the following principles:

- **Encapsulation of UI Modules:**
  - UI elements, local state, event handlers, and backend API calls of a UI component **must be encapsulated within a dedicated module**.
  - Example: The Metro Line News Carousel component must be structured in a dedicated **carousel** folder. This folder should contain:
    - **Backend API calls** for retrieving announcement images.
    - **Local state** for managing the image list and the current slide.
    - **Event handlers** for navigating between slides.
    - **The JSX/HTML structure** of the component.

- This ensures that the component is **self-contained and reusable** in different parts of the project or future projects.
  - **Separation of Concerns (SoC):**
    - Each UI module must **separate logic into distinct files**:
      1. **Component file** (.jsx / .tsx): Defines the UI structure.
      2. **State and Hooks file**: Manages local state and event handlers.
      3. **API Services file**: Handles backend API communication.
  - **Reusability Across the System:**
    - Common UI components must be designed as **shared components** and placed in a shared or common directory.
    - Reusable API services must be stored in a central services module.
    - Utility functions must be abstracted into a utils directory.
- 

### III.G.2. Backend Architecture

In addition to requirement II.G.2, the **Spring Boot backend must follow a modular monolith approach**, ensuring **clear separation of concerns** and **controlled access between modules**. Each module's interface and DTO shall be structured as follows:

#### 1. External Interface

- Defines functions that **can be accessed by other modules**.
- Typically implemented as **Spring Services** with **@Component** or **@Service** annotations.
- Example: A **TicketModule** exposes a method `purchaseTicket()` that can be called from **PaymentModule**.

#### 2. Internal Interface

- Contains functions **only accessible within the same module**.
- Internal logic should be encapsulated to prevent direct access from external modules.

- Example: The **TicketModule** contains an `internalCalculateFare()` method that is only available within the module.

### 3. External DTOs (Data Transfer Objects)

- Defines **DTOs shared between different modules** for structured communication.
- Example: A **TicketResponseDTO** is used to transfer ticket purchase data from TicketModule to PaymentModule.

### 4. Internal DTOs

- Defines **DTOs restricted to internal module use**.
- Example: A **TicketValidationDTO** is used **only within the TicketModule** for processing ticket validation logic.

By enforcing this **modularized frontend and backend architecture**, PAWA and OPWA will achieve:

- **High reusability** of components and services.
- **Better maintainability** by keeping modules independent.
- **Improved scalability** to support future features without disrupting existing functionality.

---

## III.OP. OPWA Advance System Requirements

### III.OP.1. Allowing Adding Images of National ID and Student ID

OPWA shall allow staff to upload the image of their National ID card to enable staff identification.

- The storage size of the image must be less than 5 MB.
- Each Operator must upload two images to present the front and back side of the card

### III.OP.2. Allow Staff to Login with Google Account

Instead of implementing the authentication methods specified in I.OP.2 and II.OP.2, **OPWA must support authentication via Google Accounts**. This feature is introduced to streamline security management and leverage Google's email services for operational efficiency.

- **OAuth 2.0 Integration:** OPWA backend must implement the OAuth 2.0 protocol to securely authenticate staff via their Gmail address.
- **Access Validation:** Only staff whose Gmail address and account data are created by the Admin can login to the system. Any email address not created by the Admin must be rejected.

OPWA are still required to validate the JWT token created by the PAWA system for any requests from PAWA, as stated in **II.OP.2**.

---

### III.OP.3. Advance Metro Lines Search API

In addition to the requirements specified in **II.OP.3**, OPWA must enhance passengers experience by integrating a **Map API** for visualizing metro routes and supporting cross-line trip searches. The system shall implement the following features:

- **Offer Search by Departure Time:**
  - When passengers search for train routes, OPWA must display the **next three upcoming trips** matching the search criteria of the passenger. The trip schedule shall be calculated using **existing metro line schedules**, including **first departure time and train frequency**.
- **Cross-Line Trip Search:**
  - OPWA must allow passengers to search for trips between **any two stations within the Metro Network**, even if they are on **different metro lines**.
  - If a journey requires **switching between multiple metro lines**, the system shall display the necessary transfer stations and estimated transfer times.
  - For example, a passenger searching for a trip from **Suoi Tien Station (Line 1) to Nguyen Van Linh Station (Line 4)** should receive a route that includes **transfer points and estimated travel durations**.

By implementing **real-time schedule-based searching** and **crossline trip support**, PAWA significantly enhances trip planning for passengers, offering dynamic, **intuitive, and highly informative** commuting experience

### III.OP.4. Route Visualization on Map API

OPWA must assist Operator to view the trajectory of any Metro Line in the map to ensure the metro line is functioning correctly on the destined path.

- OPWA shall **display the stations on an interactive map** to help operators view the path of Metro Lines in the system.
- The system is **not required to show direct connections between stations** due to the metro lines' incomplete status in any Map API. Instead, **individual station locations** should be shown as markers.
- The **start station and end station of the searched trip must be clearly highlighted** on the map.

By implementing **map-based route visualization**, and **cross-line trip support**, OPWA significantly supports the monitoring of Metro Line paths, offering dynamic, **intuitive, and highly informative** status report to Operators.

---

### III.OP.4. Anonymize Sensitive Staff Data

OPWA must not store **sensitive National ID and Phone Number information** in plaintext. To enhance security and comply with industry standards, **National ID and Phone Number** must be **securely stored** using encryption or hashing, depending on the data type and intended use.

---

### III.OP.5. Re-upload National ID and/or Student ID Proof

PAWA shall allow passengers to upload updated images of their **National ID** or **Student ID** in case they are issued a new card. The system must enforce the following requirements to ensure proper document submission:

- **File Size Limitation:** Each uploaded image must not exceed **5 MB** to optimize storage and loading performance.
- **Dual-Sided Submission:** For each card type, passengers must upload **two images**:
  - **Front side** of the card
  - **Back side** of the card
- **Format Validation:** The uploaded images must be in **JPEG or PNG** format to ensure compatibility.

- **Immediate Preview:** The system must display a **preview of uploaded images** before submission, allowing passengers to verify their documents.

By implementing these requirements, PAWA ensures that passengers can seamlessly update their identification documents while maintaining **data integrity, usability, and security**.

---

### III.OP.6. Dynamic, Informative, and Persisted Ticket Cart

In addition to the requirements specified in **II.OP.6**, the **Ticket Cart** must provide the following advanced features to enhance Ticket Agent experience, usability, and security:

- **Adjustable Ticket Quantities:** Ticket Agent must be able to increase or decrease the quantity of each ticket type within the cart dynamically. Changes in quantity must instantly update the subtotal and total price displayed.
- **Real-Time Cart Overview:** The Ticket Cart label, which is visible on all Ticket Agent pages, must dynamically display the **total number of tickets** and the **cumulative price** of the cart's contents. If the cart is empty, only the cart icon must be displayed.
- **Persistent Data Across Sessions:** The cart contents must remain intact even after the page is refreshed or the passenger navigates away from OPWA.
- **Secure Storage Implementation:** If browser storage (such as **localStorage** or **IndexedDB**) is used to store cart data, it must be **secured properly**. Specifically:
  - Passenger ID **must not** be stored in plaintext.
  - Sensitive identifiers must be **hashed or encrypted** to prevent unauthorized access or data tampering.

By implementing these features, the **Ticket Cart** will offer a **dynamic, informative, and resilient** ticket purchasing experience, ensuring data persistence while maintaining security best practices.

---

### III.OP.7. Partition and Shard Staff Data

To improve the scalability and performance of staff data management, OPWA must implement partitioning and sharding mechanisms as follows:

## 1. Staff Partitioning by Employment Status

- Staff records must be **partitioned** into two categories:
  - **Active Staff**: Employees currently working for HCMC Metro.
  - **Inactive Staff**: Employees who have left the organization.
- Inactive staff records must be stored in a separate partition to improve query performance for active staff operations.

## 2. Sharding Strategy for Active/Inactive Staff Data

- Within each partition (**Active** and **Inactive** staff), data must be **sharded** to distribute workload efficiently.
- OPWA team shall **determine an optimal sharding key** that balances read and write performance while supporting staff-related operations such as authentication, authorization, and profile management.
- The chosen sharding strategy must minimize cross-shard queries and ensure high availability and fault tolerance.

This approach enhances system performance, ensuring fast data retrieval for active or past employees in a structured and efficient manner.

# IV.OPAC. OPWA API Consumption Requirements

The OPWA system must consume the following API endpoints provided by the PAWA system to ensure seamless integration for **Metro Line Management** and **On-site Ticket Purchasing Services**.

## IV.OPAC.1. Payment API

- OPWA must use the **PAWA Payment API** to execute ticket payment transactions on behalf of passengers.
- After initiating a payment request, OPWA must receive and process the **transaction status** (success or failure) from the PAWA system.
- OPWA must not store or process payment credentials but must rely entirely on the **PAWA Payment API** for handling payments securely.

## IV.OPAC.2. Ticket Types API

- OPWA must retrieve the **list of available ticket types**, including their **names and prices**, from the PAWA system.
- OPWA must not maintain a separate ticket pricing database but instead fetch up-to-date ticket information directly from the PAWA system.

## V.OPAP. OPWA API Provision Requirements

To ensure seamless integration between OPWA and PAWA, OPWA must provide the following API endpoints. These APIs will allow PAWA to retrieve essential metro line data and notify passengers in real time about unexpected route closures or suspensions.

### V.OPAP.1. Metro Line Data Search and Listing API

OPWA must expose API endpoints that enable PAWA to retrieve metro line details and available trips based on passenger search criteria. The API shall adhere to the following specifications:

- **Authentication:** Not required; available to both guests and registered passengers.
- **Response Data:**
  - **Metro Line Information:** Includes Line Name, Start Station, End Station, Intermediate Stations, Start Time, and End Time.
  - **Route Suspension Details:** Indicates whether specific metro trips or segments are suspended and provide relevant explanations (e.g., maintenance, emergencies).

---

### V.OPAP.2. Real-Time Metro Line Suspension Notification API

To enhance passenger awareness and minimize travel disruptions, OPWA must provide real-time notifications to PAWA when a metro line or segment is suspended. The API shall adhere to the following specifications:

- **Trigger Mechanism:** Whenever an Operator marks a metro line or a segment as suspended in OPWA, an event-driven notification must be sent to PAWA.

- **Data Payload:**
  - Metro Line and Station details.
  - Reason for suspension (e.g., emergency, maintenance).
  - Expected resolution time (if available).
- **Delivery Method:**
  - PAWA must receive this data and immediately display an in-app notification to all affected passengers.
  - The API should ensure low-latency updates to provide real-time awareness.

## The End